

Controle de Vencimento

Entrega 3 - Arquitetura do Sistema

Projeto Integrador

Grupo: 7

Curso: Análise e Desenvolvimento de Sistemas

Disciplina: Projeto Integrador

Integrantes

Tiago Boccato Barbosa - RA: 1726105234

Jennifer Souza de Oliveira - RA: 926112013

Paloma Dalila de Jesus Duarte Amorim Santos - RA: 1726107086

Ariana Barbosa Costa - RA: 926112800

Gabriel Celestino Araujo - RA: 925208181

1. Introdução

O Controle de Vencimento é um sistema SaaS voltado ao cadastro, acompanhamento e controle de produtos sensíveis, como alimentos, medicamentos e itens que possuem data de validade.

A proposta do sistema é auxiliar usuários e pequenos estabelecimentos na organização de produtos e no acompanhamento dos prazos de vencimento. Por meio do registro dos itens e da emissão de alertas, a solução contribui para reduzir desperdícios, evitar perdas e melhorar o controle dos produtos armazenados.

A interface do sistema foi representada por meio do protótipo desenvolvido no Figma, sendo considerada na arquitetura como a camada de apresentação responsável pela interação com o usuário.

2. Objetivo da arquitetura

O objetivo desta documentação é apresentar a arquitetura proposta para o sistema Controle de Vencimento, contemplando a visão geral da solução, os principais elementos do sistema, os diagramas C4 e a modelagem de dados relacional e não relacional.

A arquitetura foi organizada em três níveis do modelo C4: Contexto (C1), Container (C2) e Component (C3).

3. Visão geral da solução

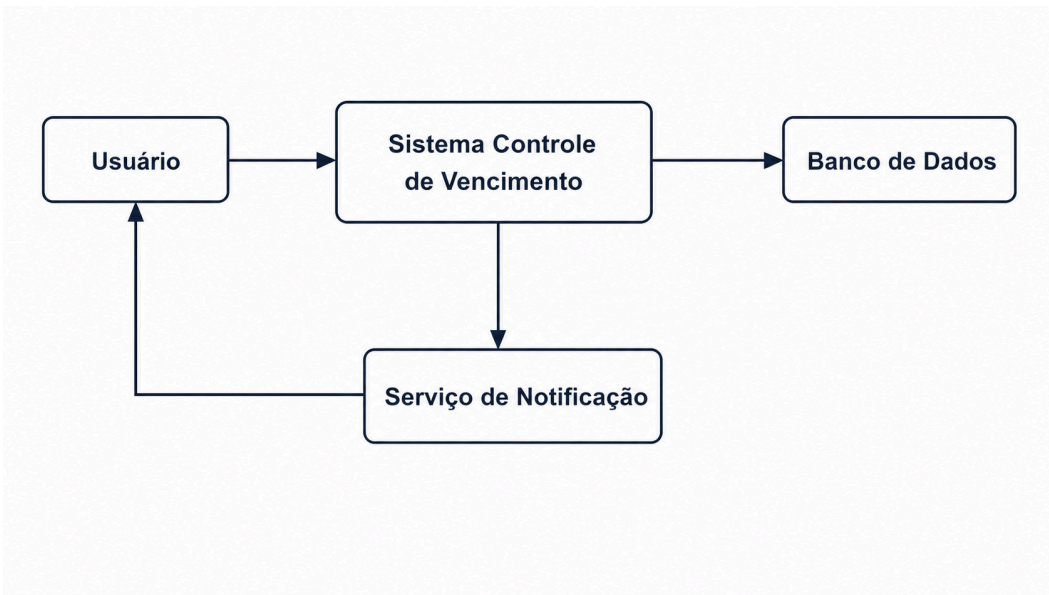
A solução proposta é composta por uma interface web representada no Figma, uma API Back-end responsável pelas regras do sistema, um banco de dados MySQL para armazenamento das informações principais e um serviço de notificação para envio de alertas aos usuários.

O fluxo principal do sistema envolve o cadastro de usuários, categorias e produtos. A partir da data de validade informada, o sistema registra alertas relacionados aos produtos próximos do vencimento. Esses alertas podem gerar notificações, permitindo que o usuário acompanhe os itens que exigem atenção.

De forma complementar, a arquitetura apresenta o uso previsto do MongoDB para histórico de alertas enviados e do Redis para cache temporário de alertas.

4. Diagrama C4 - Contexto (C1)

O diagrama de contexto apresenta o sistema em uma visão de alto nível, mostrando a interação entre o usuário, o Sistema Controle de Vencimento, o banco de dados e o serviço de notificação.

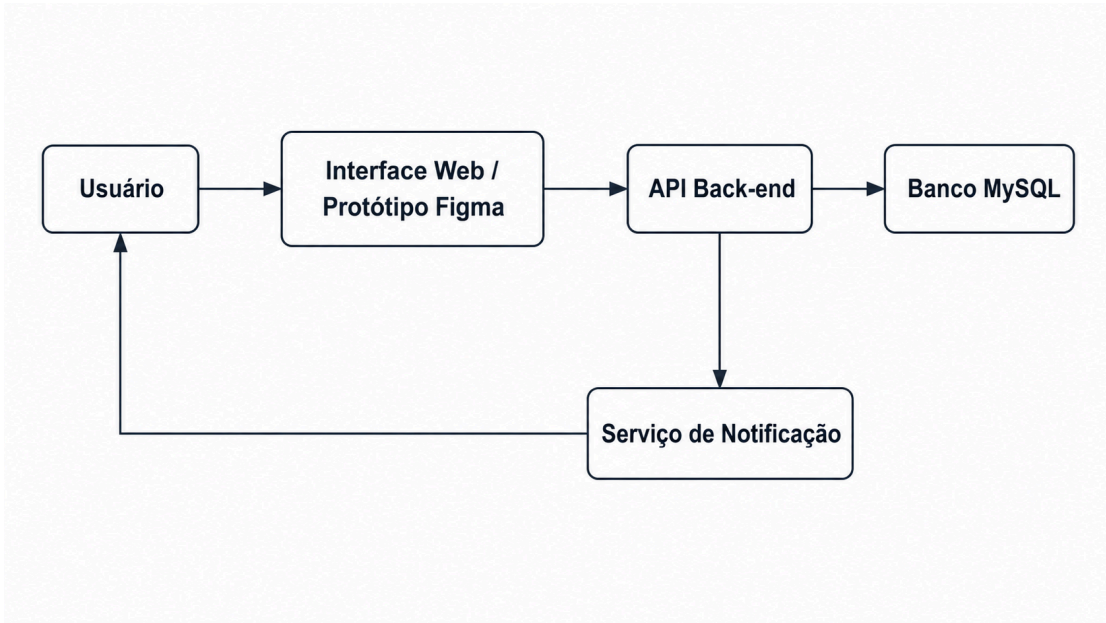


Descrição dos elementos do C1

Elemento	Descrição
Usuário	Pessoa ou estabelecimento que utiliza o sistema para cadastrar produtos e acompanhar vencimentos.
Sistema Controle de Vencimento	Aplicação responsável por organizar produtos, datas de validade e alertas.
Banco de Dados	Local onde são armazenadas as informações principais do sistema.
Serviço de Notificação	Recurso responsável pelo envio de avisos relacionados aos produtos próximos do vencimento.

5. Diagrama C4 - Container (C2)

O diagrama de container apresenta as principais partes da solução. A interface web, representada pelo protótipo no Figma, corresponde à camada de apresentação. A API Back-end representa a camada de aplicação, responsável por processar as regras do sistema e acessar o banco de dados.

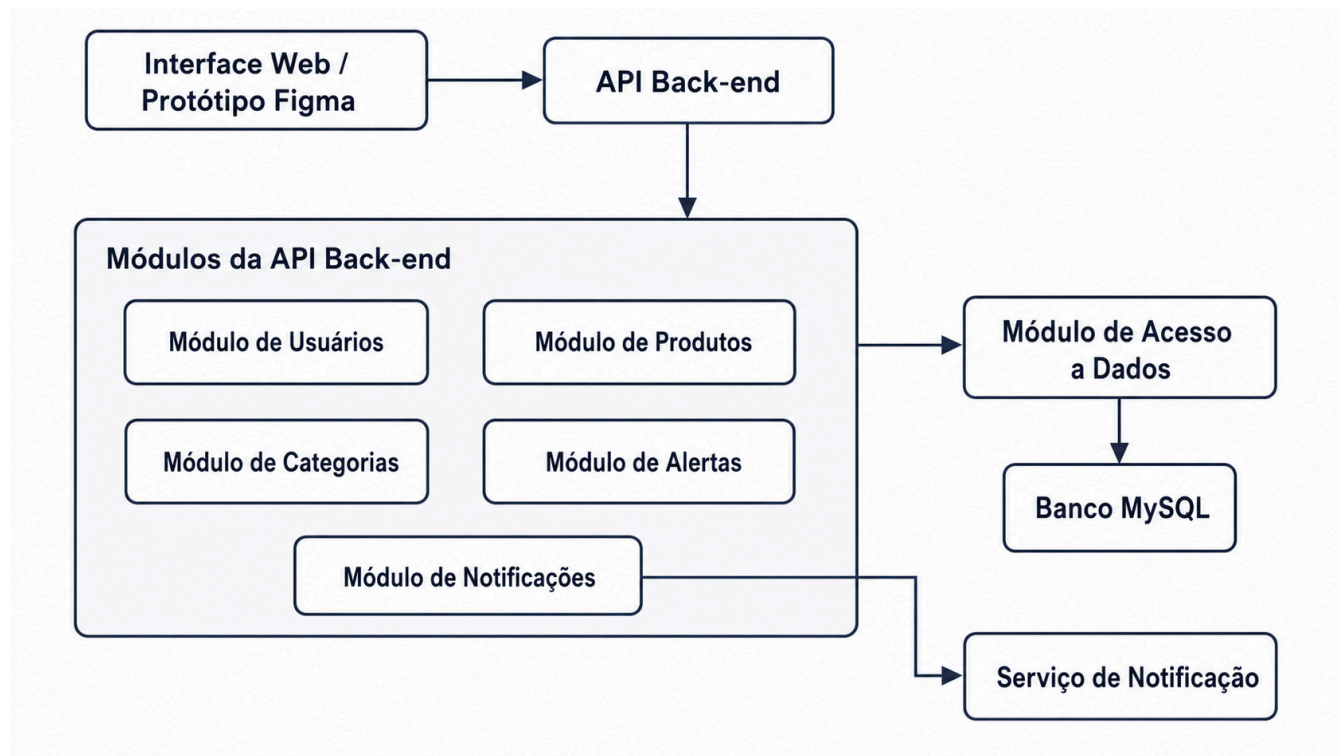


Descrição dos containers do C2

Container	Responsabilidade
Interface Web / Protótipo Figma	Representa a camada de apresentação e a interação do usuário com o sistema.
API Back-end	Centraliza regras de negócio, validações, cadastro de produtos e geração de alertas.
Banco MySQL	Armazena os dados principais do sistema em tabelas relacionadas.
Serviço de Notificação	Envia avisos relacionados aos produtos próximos do vencimento.

6. Diagrama C4 - Component (C3)

O diagrama de componentes apresenta uma visão interna da API Back-end, destacando os principais módulos responsáveis pelas funcionalidades do sistema.



Descrição dos componentes do C3

Componente	Responsabilidade
Módulo de Usuários	Responsável pelas operações relacionadas ao cadastro, login e dados do usuário.
Módulo de Produtos	Responsável pelo cadastro, edição, exclusão, busca e listagem de produtos.
Módulo de Categorias	Responsável pela organização dos produtos por categoria.
Módulo de Alertas	Responsável por identificar produtos próximos do vencimento e registrar alertas.
Módulo de Notificações	Responsável por organizar o envio ou registro das notificações geradas.
Módulo de Acesso a Dados	Responsável pela comunicação entre a API Back-end e o banco de dados.

7. Diagrams as Code

Os diagramas da arquitetura foram elaborados em Mermaid, uma ferramenta que permite representar diagramas por meio de código textual. Os códigos dos diagramas estão disponíveis no repositório GitHub nos seguintes arquivos:

- docs/entrega-3/diagrams/c1-contexto.mmd
- docs/entrega-3/diagrams/c2-container.mmd
- docs/entrega-3/diagrams/c3-componentes.mmd

8. Modelagem relacional - MySQL

O banco relacional MySQL foi definido para armazenar os dados principais da aplicação. A modelagem proposta possui cinco tabelas: usuarios, categorias, produtos, alertas e notificacoes.

O script SQL completo está disponível em docs/entrega-3/sql/schema.sql.

Tabela	Campos principais	Relacionamentos
usuarios	id, nome, email, senha, created_at	Um usuário pode possuir vários produtos.
categorias	id, nome, descricao, created_at	Uma categoria pode estar associada a vários produtos.
produtos	id, usuario_id, categoria_id, nome, quantidade, data_validade, created_at	Produto pertence a um usuário e a uma categoria.
alertas	id, produto_id, tipo_alerta, mensagem, data_alerta, created_at	Um produto pode gerar vários alertas.
notificacoes	id, alerta_id, canal, status, mensagem, data_envio, created_at	Um alerta pode gerar várias notificações.

Modelo relacional textual

- usuarios (id, nome, email, senha, created_at)
- categorias (id, nome, descricao, created_at)
- produtos (id, usuario_id, categoria_id, nome, quantidade, data_validade, created_at)
- alertas (id, produto_id, tipo_alerta, mensagem, data_alerta, created_at)
- notificacoes (id, alerta_id, canal, status, mensagem, data_envio, created_at)

9. Modelagem NoSQL - MongoDB

O MongoDB foi considerado para armazenar o histórico de alertas enviados. Esse tipo de registro pode guardar informações do alerta, do produto, do usuário e do envio realizado.

Campos previstos no documento: `usuario_id`, `produto_id`, `nome_produto`, `mensagem`, `canal`, `status`, `data_envio` e `detalhes`.

O modelo de documento MongoDB completo está disponível em `docs/entrega-3/nosql/mongodb-historico-alertas.json`.

10. Modelagem Redis

O Redis foi considerado para armazenamento temporário de alertas de vencimento. A estrutura em chave e valor pode manter informações de consulta rápida, como alertas recentes ou produtos próximos do vencimento.

Item	Estrutura resumida
Chave	<code>alerta:produto:15</code>
Valor	Produto vence em 3 dias
Tempo de expiração	86400 segundos

A estrutura Redis completa está disponível em `docs/entrega-3/nosql/redis-cache-alertas.txt`.

11. Arquivos disponíveis no GitHub

Os arquivos técnicos da Entrega 3 estão organizados no repositório GitHub:

- `docs/entrega-3/diagrams/c1-contexto.mmd`
- `docs/entrega-3/diagrams/c2-container.mmd`
- `docs/entrega-3/diagrams/c3-componentes.mmd`
- `docs/entrega-3/sql/schema.sql`
- `docs/entrega-3/nosql/mongodb-historico-alertas.json`
- `docs/entrega-3/nosql/redis-cache-alertas.txt`